Dr. Tilápia

PLANO DE IMPLEMENTAÇÃO - Next.js + FastAPI

Migração Frontend e Integração com Supabase Vector Store

06 de abril de 2026

---

1. Visão Geral

Este documento detalha o plano de implementação para a transição do frontend do projeto Dr. Tilápia para uma arquitetura moderna e robusta, utilizando**Next.js**. O objetivo é substituir o frontend atual baseado em Reflex, que apresentou desafios de compatibilidade e estabilidade, por uma solução mais madura e escalável. O backend existente, construído com**FastAPI**, será integralmente aproveitado, garantindo a continuidade das funcionalidades e a otimização do tempo de desenvolvimento. Além disso, a solução de armazenamento vetorial será consolidada no **Supabase Vector Store**, substituindo o FAISS e simplificando a infraestrutura.

A nova arquitetura visa proporcionar maior performance, melhor experiência de desenvolvimento, facilidade de manutenção e escalabilidade, aproveitando as melhores práticas do ecossistema React e Next.js, enquanto mantém a eficiência e a robustez do backend FastAPI e a flexibilidade do Supabase.

2. Arquitetura Atual do Backend (FastAPI)

O backend do Dr. Tilápia, desenvolvido em FastAPI, já possui uma estrutura bem definida e funcionalidades essenciais implementadas. Esta camada será mantida intacta, servindo como a espinha dorsal da lógica de negócios e integração com o banco de dados e serviços de IA.

2.1. Estrutura e Endpoints Implementados

O coração do backend reside no arquivo main.py, que orquestra os seguintes endpoints:

• **GET /health:**Um endpoint simples para verificar a saúde e disponibilidade da API.

• **POST /auth/login:**Responsável pela autenticação de usuários. Este endpoint integra-se nativamente com o serviço de autenticação do Supabase, utilizando o método sign_in_with_password para validar credenciais e retornar um token JWT.

• **POST /admin/upload:**Permite que usuários administradores façam o upload de documentos PDF. A função ingest_pdf é acionada para processar o arquivo, extrair texto, dividi-lo em chunks, gerar embeddings e armazená-los no Supabase Vector Store.

• **POST /consultoria/chat:**O principal endpoint para a funcionalidade de RAG (Retrieval Augmented Generation). Recebe uma consulta do usuário, utiliza o ConversationalRetrievalChain para buscar informações relevantes no banco de vetores e gerar uma resposta contextualizada usando um Large Language Model (LLM).

2.2. Componentes do RAG Service

O serviço de RAG é composto por:

• **SupabaseVectorStore:**Conecta-se à tabela documents no Supabase, que armazena os embeddings dos documentos. Esta é a principal fonte de conhecimento para o RAG.

• **OpenAIEmbeddings:**Utiliza o modelo text-embedding-3-small da OpenAI para gerar representações vetoriais (embeddings) de textos, tanto para os documentos quanto para as consultas dos usuários.

• **ChatOpenAI:**Emprega o modelo GPT-4o da OpenAI como o LLM para gerar respostas em linguagem natural, com base nas informações recuperadas do Supabase Vector Store.

2.3. Autenticação e Banco de Dados

A autenticação é gerenciada de forma nativa pelo Supabase, através do método sign_in_with_password. As tabelas essenciais no Supabase para o funcionamento do sistema são:

Tabela	Descrição
users	Gerenciada pelo serviço de autenticação do Supabase, armazena informações dos usuários.
documents	Armazena os chunks de texto dos documentos processados, juntamente com seus embeddings vetoriais (coluna vector).

3. Novo Frontend Next.js - Estrutura

O novo frontend será construído com Next.js, utilizando o**App Router**para uma estrutura de rotas moderna e eficiente. A escolha do Next.js proporciona renderização no servidor (SSR), geração de sites estáticos (SSG) e renderização incremental estática (ISR), otimizando a performance e a experiência do usuário.

3.1. Estrutura de Rotas e Componentes Principais

A aplicação será organizada com as seguintes rotas principais:

• **/auth:**Contém as páginas relacionadas à autenticação, como login e registro.

• **/main:**Abriga as funcionalidades principais da aplicação, acessíveis após o login.

Os componentes essenciais a serem desenvolvidos incluem:

• **Login:**Página de entrada para autenticação de usuários.

• **Hub:**Dashboard principal da aplicação, onde o usuário pode navegar para outras funcionalidades.

• **Consultoria (Chat):**Interface para interação com o serviço de RAG.

• **Admin (Upload):**Página para administradores fazerem o upload de novos documentos.

3.2. Gerenciamento de Estado e Comunicação com API

Para o gerenciamento de estado, será utilizada uma abordagem híbrida:

• **TanStack Query (React Query):**Ideal para gerenciamento de estado de servidor (server state), como cache de dados, sincronização e requisições assíncronas aos endpoints do FastAPI. Garante que os dados estejam sempre atualizados e otimiza a performance.

• **Zustand:**Um gerenciador de estado leve e flexível para o estado do cliente (client state), como o estado de autenticação do usuário, configurações de UI e dados temporários.

A comunicação com o backend FastAPI será feita através de um**Client API**dedicado, que encapsulará as chamadas HTTP e o tratamento de tokens de autenticação.

4. Fluxo de Dados (CRÍTICO)

O fluxo de dados entre o frontend Next.js e o backend FastAPI é crucial para o funcionamento do sistema. Abaixo, detalhamos o processo, com foco na autenticação e nas interações com o RAG Service.

> "O fluxo de dados é a espinha dorsal da aplicação, garantindo que as informações transitem de forma segura e eficiente entre as camadas."

4.1. Diagrama de Fluxo de Dados

```mermaid
```

```
graph TD

A[Usuário] -- Acessa --> B(Next.js Frontend: http://localhost:3000)

B -- Preenche Login --> C{Página de Login}

C -- POST /auth/login --> D(FastAPI Backend: http://localhost:8000)

D -- Chama sign_in_with_password --> E[Supabase Auth]

E -- Retorna JWT --> D

D -- Retorna JWT --> C

C -- Armazena JWT --> F[Next.js: localStorage/cookies]

C -- Redireciona --> G(Next.js Frontend: /main/consultoria)

G -- Envia Mensagem + JWT --> H{Componente ChatConsultoria}

H -- POST /consultoria/chat --> D

D -- Valida JWT --> E

D -- Query Embedding --> I[OpenAI Embeddings]

I -- Vetor da Query --> D

D -- match_documents RPC --> J[Supabase Vector Store: tabela 'documents']

J -- Chunks Relevantes --> D

D -- Geração de Resposta --> K[ChatOpenAI (GPT-4o)]

K -- Resposta --> D

D -- Resposta --> H

H -- Exibe Resposta --> G
```

4.2. Detalhamento do Fluxo

1. O usuário acessa a aplicação Next.js através de http://localhost:3000.

2. Na página de login, o usuário insere suas credenciais (email e senha).

3. O frontend Next.js envia uma requisição**POST**para o endpoint http://localhost:8000/auth/login do FastAPI.

4. O FastAPI, por sua vez, interage com o serviço de autenticação do Supabase para validar as credenciais. Se bem-sucedido, o Supabase retorna um token JWT.

5. O FastAPI repassa o token JWT para o frontend Next.js.

6. O Next.js armazena este token (preferencialmente em localStorage ou cookies seguros) para futuras requisições autenticadas.

7. Após o login, o usuário é redirecionado para a página principal (ex: /main/consultoria).

8. Quando o usuário envia uma mensagem no chat, o componente ChatConsultoria do Next.js envia uma requisição**POST**para http://localhost:8000/consultoria/chat, incluindo o token JWT no cabeçalho de autorização.

9. O FastAPI valida o token JWT. Se válido, ele inicia o processo de RAG: * A consulta do usuário é convertida em um embedding vetorial usando**OpenAI Embeddings**. * Este vetor é usado para chamar a função match_documents (RPC) no**Supabase Vector Store**, que busca os chunks de documentos mais relevantes na tabela documents. * Os chunks recuperados são passados para o**ChatOpenAI (GPT-4o)**, que gera uma resposta contextualizada.

10. A resposta gerada pelo LLM é retornada pelo FastAPI para o frontend Next.js, que a exibe na interface do chat.

5. Integração Específica: Supabase Vector Store

A utilização do Supabase como banco de dados relacional e, crucialmente, como Vector Store, simplifica a arquitetura e o deployment. O backend FastAPI já está configurado para interagir com ele.

5.1. Conexão do RAGService

O RAGService no backend FastAPI é configurado para se conectar diretamente à tabela documents do Supabase. Esta tabela deve conter uma coluna do tipo vector (extensão pgvector) para armazenar os embeddings.

5.2. Função match_documents RPC

Para otimizar a busca vetorial, uma função RPC (Remote Procedure Call) chamada match_documents é criada no Supabase. Esta função SQL recebe o vetor da consulta e retorna os chunks de documentos mais semanticamente próximos, utilizando operadores de similaridade vetorial.

5.3. Processo de ingest_pdf (Upload de Documentos)

Quando um PDF é enviado via POST /admin/upload:

1. **LoadPDF:**O arquivo PDF é lido e seu conteúdo textual é extraído.

2. **Split:**O texto é dividido em pedaços menores (chunks) para melhor granularidade na busca.

3. **Add Metadata:**Metadados relevantes (ex: nome do arquivo, página) são associados a cada chunk.

4. **Vetoriza:**Cada chunk é transformado em um embedding vetorial usando OpenAIEmbeddings.

5. **Salva no Supabase:**Os chunks de texto, seus metadados e seus embeddings são inseridos na tabela documents do Supabase.

5.4. Processo de Retrieval (Busca de Informações)

Durante uma consulta no chat (POST /consultoria/chat):

1. A consulta do usuário é convertida em um embedding vetorial.

2. Este embedding é enviado como parâmetro para a função match_documents RPC no Supabase.

3. A função RPC executa a busca de similaridade vetorial e retorna os chunks de documentos mais relevantes.

4. Esses chunks são então utilizados pelo LLM para gerar a resposta final.

6. Configuração do Docker Compose

O docker-compose.yml será atualizado para incluir o serviço do Next.js (frontend) e garantir a comunicação adequada com o FastAPI (backend) através de uma rede compartilhada.

Serviço	Descrição	Configuração Chave
api	Backend FastAPI	build: ./backendports: "8000:8000"networks: - tilapia_net
frontend	Frontend Next.js	build: ./frontendports: "3000:3000"environment: NEXT_PUBLIC_API_URL=http://api:8000netwo - tilapia_net

Observação:O NEXT_PUBLIC_API_URL no serviço frontend aponta para http://api:8000. Dentro da rede Docker (tilapia_net), os serviços podem se comunicar usando seus nomes de serviço como hostname. Para acesso externo, o Next.js será acessível via http://localhost:3000 e o FastAPI via http://localhost:8000.

7. Passo a Passo Prático (4 Fases)

A migração do frontend será dividida em quatro fases distintas, com estimativas de tempo para cada uma, totalizando aproximadamente 3 a 4 semanas de trabalho intensivo.

Fase	Duração Estimada	Tarefas Principais
FASE 1: Setup Next.js + Estrutura	3 dias	- Inicializar projeto Next.js com TypeScript.- Instalar dependências essenciais (TailwindCSS, TanStack Query, Zustand).- Configurar estrutura de pastas (App Router, lib/api, hooks).- Criar layouts básicos e navegação.
FASE 2: Componentes de UI	7 dias	- Desenvolver a página de Login (/auth/login).- Desenvolver a página Hub (/main).- Desenvolver a interface de Consultoria (Chat) (/main/consultoria).- Desenvolver a interface de Admin (Upload) (/main/admin).- Foco na construção da UI estática, sem integração com a API ainda.

FASE 3: Integração com APIs FastAPI	7 dias	- Implementar o cliente API (lib/api.ts) para todos os endpoints.- Desenvolver o hook useAuth para gerenciamento de JWT.- Conectar o componente de Login ao endpoint /auth/login.- Conectar o componente de Chat ao endpoint /consultoria/chat.- Conectar o componente de Upload ao endpoint /admin/upload.- Implementar gerenciamento de estado com TanStack Query e Zustand para dados da API.
FASE 4: Testes e Refinamento	7 dias	- Realizar testes unitários para componentes e hooks.- Implementar testes end-to-end (E2E) com Cypress ou Playwright.- Ajustar a responsividade da interface para diferentes dispositivos.- Otimizar performance (carregamento, renderização).- Revisão de segurança e tratamento de erros.

8. Código de Exemplo (ATUALIZADO)

A seguir, exemplos de código que ilustram a implementação do cliente API, do hook de autenticação e dos componentes de chat e upload, refletindo a integração com o backend FastAPI e o Supabase.

8.1. lib/api.ts - Cliente API

typescript
// lib/api.ts
import axios from 'axios';
const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000';

const api = axios.create({ baseURL: API_BASE_URL, headers: { 'Content-Type': 'application/json', }, });

// Interceptor para adicionar o token JWT api.interceptors.request.use((config) => { const token = localStorage.getItem('supabase.auth.token'); // Ou de cookies if (token) { config.headers.Authorization = Bearer ${JSON.parse(token).access_token}; } return config; }, (error) => Promise.reject(error));

export const authApi = { login: async (email: string, password: string) => { const response = await api.post('/auth/login', { email, password }); return response.data; // Retorna o JWT do Supabase }, };

export const consultoriaApi = { chat: async (message: string) => { const response = await api.post('/consultoria/chat', { message }); return response.data; // Retorna a resposta do RAG }, };

export const adminApi = { uploadPdf: async (file: File) => { const formData = new FormData(); formData.append('file', file); const response = await api.post('/admin/upload', formData, { headers: { 'Content-Type': 'multipart/form-data', }, }); return response.data; // Retorna status do upload }, };

export default api;

8.2. hooks/useAuth.ts - Hook de Autenticação

// hooks/useAuth.ts

import { create } from 'zustand';

import { authApi } from '@/lib/api'; // Ajuste o caminho conforme sua estrutura

import { useRouter } from 'next/navigation';

interface AuthState {

isAuthenticated: boolean;

token: string	**null; **

user: any	**null; // Pode ser tipado com mais detalhes
	**

login: (email: string, password: string) => Promise<void>;

logout: () => void;

initializeAuth: () => void;

}

8.3. app/main/consultoria/page.tsx - Componente ChatConsultoria

// app/main/consultoria/page.tsx

'use client';

import React, { useState, useEffect } from 'react';

import { consultoriaApi } from '@/lib/api'; // Ajuste o caminho

import { useAuth } from '@/hooks/useAuth'; // Ajuste o caminho

import { useMutation } from '@tanstack/react-query';

interface Message {

id: number;

text: string;

sender: 'user'	**'ai';**

}

export default function ConsultoriaPage() {

const { isAuthenticated } = useAuth();

const [messages, setMessages] = useState<Message[]>([]);

const [inputMessage, setInputMessage] = useState('');

const [messageIdCounter, setMessageIdCounter] = useState(0);

const chatMutation = useMutation({

mutationFn: consultoriaApi.chat,

onSuccess: (data) => {

setMessages((prev) => [

...prev,

{ id: messageIdCounter + 1, text: data.response, sender: 'ai' },

]);

setMessageIdCounter((prev) => prev + 1);

},

onError: (error) => {

```
'ai' },
]);
setMessageIdCounter((prev) => prev + 1);
},
});
const handleSendMessage = async () => {
if (inputMessage.trim() === '') return;
const userMessage: Message = {<br/>
id: messageIdCounter,<br/>
text: inputMessage,<br/>
sender: 'user',
};
setMessages((prev) => [...prev, userMessage]);
setMessageIdCounter((prev) => prev + 1);
setInputMessage('');
chatMutation.mutate(inputMessage);
};
if (!isAuthenticated) {
return <p>Por favor, faça login para acessar a consultoria.</p>;
}
return (
<div className="flex flex-col h-screen bg-gray-100">
<header className="bg-blue-600 text-white p-4 text-lg font-bold">
Consultoria Dr. Tilápia
</header>
<div className="flex-1 overflow-y-auto p-4 space-y-4">
{messages.map((msg) => (
<div
```

disabled:opacity-50"

disabled={chatMutation.isPending}

>

Enviar

</button>

</footer>

);

}

8.4. app/main/admin/page.tsx - Componente Upload

// app/main/admin/page.tsx

'use client';

import React, { useState } from 'react';

import { adminApi } from '@/lib/api'; // Ajuste o caminho

import { useAuth } from '@/hooks/useAuth'; // Ajuste o caminho

import { useMutation } from '@tanstack/react-query';

export default function AdminUploadPage() {

const { isAuthenticated, user } = useAuth();

const [selectedFile, setSelectedFile] = useState<File	**null>(null);**

const uploadMutation = useMutation({

mutationFn: adminApi.uploadPdf,

onSuccess: (data) => {

alert(Upload bem-sucedido: ${data.message});

setSelectedFile(null);

},

onError: (error) => {

if (!isAuthenticated	user?.role !== 'admin') {

return Acesso negado. Apenas administradores podem fazer upload. ;

}

return (

<h1 className="text-3xl font-bold mb-6 text-gray-800">Upload de Documentos PDF</h1>

<input

type="file"

accept=".pdf"

onChange={handleFileChange}

className="block w-full text-sm text-gray-500

file:mr-4 file:py-2 file:px-4

file:rounded-full file:border-0

file:text-sm file:font-semibold

file:bg-blue-50 file:text-blue-700

hover:file:bg-blue-100"

/>

{selectedFile && (

Arquivo selecionado:
{selectedFile.name}

)}

<button

onClick={handleUpload}

disabled={!selectedFile	uploadMutation.isPending}

```
className="mt-6 w-full px-4 py-2 bg-green-600 text-white rounded-lg hover:bg-green-700 disabled:opacity-50 disabled:cursor-not-allowed"

>

{uploadMutation.isPending ? 'Enviando...' : 'Fazer Upload'}

</button>

{uploadMutation.isError && (

<p className="mt-4 text-red-500">Erro: {uploadMutation.error?.message}</p>

)}

</div>

</div>

);

}
```

9. Deployment

A estratégia de deployment será otimizada para cada componente da arquitetura, aproveitando as plataformas mais adequadas para Next.js, FastAPI e Supabase.

Componente	Plataforma de Deployment	Benefícios
Frontend (Next.js)	Vercel	- Otimizado para Next.js.- Deployment contínuo (Git-based).- CDN global, edge functions, escalabilidade automática.
Backend (FastAPI)	Railway / Render	- Suporte a Docker.- Deployment contínuo.- Escalabilidade e gerenciamento de infraestrutura simplificados.

Database (PostgreSQL + Vector Store)	Supabase	- Já configurado e em uso.- Gerenciamento de banco de dados, autenticação e armazenamento de vetores em uma única plataforma.

Dr. Tilápia - Equipe de Desenvolvimento

Local e data: Brasília, 06 de abril de 2026